

Top-k & Top-p Sampling, Worked Out by Hand

Two ways to chop the unreliable tail off a language model's next-token distribution before sampling — a fixed count (top- k) versus an adaptive probability mass (top- p /nucleus) — with the truncation and renormalisation done on real numbers.

What this document does. It starts from a concrete softmax distribution over five tokens, sorts it, and then applies both truncation rules by hand: keep the top k , or keep the smallest set whose cumulative probability reaches p . In each case it shows the cut and the *renormalisation* that turns the survivors back into a valid distribution, then explains why nucleus sampling adapts to the model's confidence where top- k cannot. Numbers from Appendix A.

Prerequisite: softmax (#1) and temperature (#2). Sampling acts on the distribution those produce.

1 Why truncate at all?

A language model assigns *some* probability to every token in a 50,000-word vocabulary. Most of that tail is garbage — individually tiny, but collectively a few percent. Sample from the full distribution often enough and you will eventually draw a nonsense token, derailing the generation. Truncation deletes the tail *before* sampling, then renormalises so the kept tokens sum to one again.

Our running distribution (softmax of $\mathbf{z} = (2, 1, 0.1, -1, 3)$), sorted high-to-low:

rank token	1 mat	2 cat	3 sat	4 on	5 the
p_i	0.6343	0.2333	0.0858	0.0349	0.0116
cumulative	0.6343	0.8676	0.9535	0.9884	1.0000

Intuition

Sampling is a dartboard whose wedges are sized by probability. The tail tokens are slivers — but there are thousands of them, and the dart only has to land on one absurd sliver to ruin a sentence. Truncation simply paints over the slivers and re-inflates the big wedges to fill the board. Top- k paints over a *fixed number* of slivers; top- p paints over *whatever it takes* to leave a chosen fraction of the board.

2 Top- k : keep a fixed count

Rule. Keep the k highest-probability tokens, zero the rest, renormalise:

$$p'_i = \frac{p_i [i \in \text{Top}_k]}{\sum_{j \in \text{Top}_k} p_j}$$

Worked example, $k = 2$. Keep $\{\text{mat}, \text{cat}\}$; their mass is $0.6343 + 0.2333 = 0.8676$. Renormalise by dividing each by 0.8676:

$$p'_{\text{mat}} = \frac{0.6343}{0.8676} = 0.7311, \quad p'_{\text{cat}} = \frac{0.2333}{0.8676} = 0.2689.$$

The other three tokens now have probability 0; the model can only emit `mat` or `cat`, in a 73/27 split.

Intuition

Top- k 's weakness: k is blind to shape. When the model is *sure* (one token at 95%), $k = 40$ still drags in 39 junk tokens. When the model is *torn* (mass spread over 200 near-ties), $k = 40$ amputates 160 perfectly reasonable ones. A fixed count cannot fit a distribution whose width keeps changing token to token.

3 Top- p (nucleus): keep a fixed mass

Rule. Walk down the sorted tokens accumulating probability; stop as soon as the running sum reaches the threshold p . That smallest set — the *nucleus* — is kept and renormalised:

$$\text{nucleus} = \text{smallest prefix with } \sum p_i \geq p, \quad p'_i = \frac{p_i}{\sum_{j \in \text{nucleus}} p_j}$$

Worked example, $p = 0.9$. Read down the cumulative row:

$$0.6343 (< 0.9) \rightarrow 0.8676 (< 0.9) \rightarrow 0.9535 (\geq 0.9) \text{ stop.}$$

The nucleus is `{mat, cat, sat}` with mass 0.9535. Renormalise by 0.9535:

$$p'_{\text{mat}} = \frac{0.6343}{0.9535} = 0.6652, \quad p'_{\text{cat}} = \frac{0.2333}{0.9535} = 0.2447, \quad p'_{\text{sat}} = \frac{0.0858}{0.9535} = 0.0900.$$

Sum = 1.0000. ✓ Three tokens survive here — but that count was *discovered*, not fixed.

Mini-example • the nucleus shrinks when the model is confident

Re-temper the same logits at $T = 0.7$ (sharper). The sorted probabilities become

$$(0.7597, 0.1821, 0.0436, 0.0121, 0.0025), \quad \text{cumulative } (0.7597, 0.9418, \dots).$$

Now the running sum crosses 0.9 after just *two* tokens ($0.9418 \geq 0.9$). Same $p = 0.9$, but the nucleus shrank from 3 tokens to 2 because the model got more certain. **This is the whole point: top- p widens when the model hedges and narrows when it commits.** A fixed k could never do that.

4 Choosing between them (and together)

- **Top- k** ($k \approx 20$ –50): a hard cap on candidates. Cheap, predictable, but mis-sized whenever the distribution's width changes.
- **Top- p** ($p \approx 0.9$ –0.95): the modern default. The candidate set breathes with the model's confidence, so it neither starves a torn distribution nor floods a confident one.
- **Both at once** (common in practice): apply top- k as a safety ceiling, then top- p within it. And recall the ordering from PDF #2 — temperature reshapes *first*, then truncation cuts the reshaped tail.

Equation cheat-sheet

$$\text{top-}k: \quad p'_i = \frac{p_i [i \in \text{Top}_k]}{\sum_{j \in \text{Top}_k} p_j}$$

$$\text{top-}p: \quad p'_i = \frac{p_i [i \in \text{nucleus}]}{\sum_{j \in \text{nucleus}} p_j}$$

nucleus = smallest sorted prefix with $\sum p_i \geq p$ (set size is dynamic, not fixed)

Appendix A · Reference implementation

```
import numpy as np

def top_k(p, k):
    idx = np.argsort(-p)[:k]
    q = np.zeros_like(p); q[idx] = p[idx]
    return q / q.sum()

def top_p(p, thr):
    idx = np.argsort(-p)
    cum = np.cumsum(p[idx])
    cut = np.searchsorted(cum, thr) + 1      # smallest prefix reaching thr
    keep = idx[:cut]
    q = np.zeros_like(p); q[keep] = p[keep]
    return q / q.sum()

p = np.array([0.6343, 0.2333, 0.0858, 0.0349, 0.0116]) # mat,cat,sat,on,the
top_k(p, 2)      # mat 0.7311, cat 0.2689
top_p(p, 0.9)    # mat 0.6652, cat 0.2447, sat 0.0900
```

Internal learning material. Numbers reproducible from the appendix code. v1.0